

ПРОГРАМНАЯ ДОКУМЕНТАЦИЯ НА “ОБЛАЧНОЕ
ХРАНИЛИЩЕ КАРТИНОК” РАЗРАБОТАННЫЙ НА ОСНОВЕ

Python

Django

PyQt5

ЮЖНО-САХАЛИНСК 2026

СОДЕРЖАНИЕ

1.Введение	3
2.Техническое задание	3
3.Схема разработанной программы	3
4.Перечень и нумерация событий	4
5.Перечень и нумерация входных переменных	4
6.Перечень и нумерация выходных воздействий	4
7.Как пользоваться вашей программой	4
8.Системонезависимая часть	4
9.Листинг	4

1.ВВЕДЕНИЕ

Проект “Облачное хранилище картинок” включает в себя серверное приложение и клиентское приложение

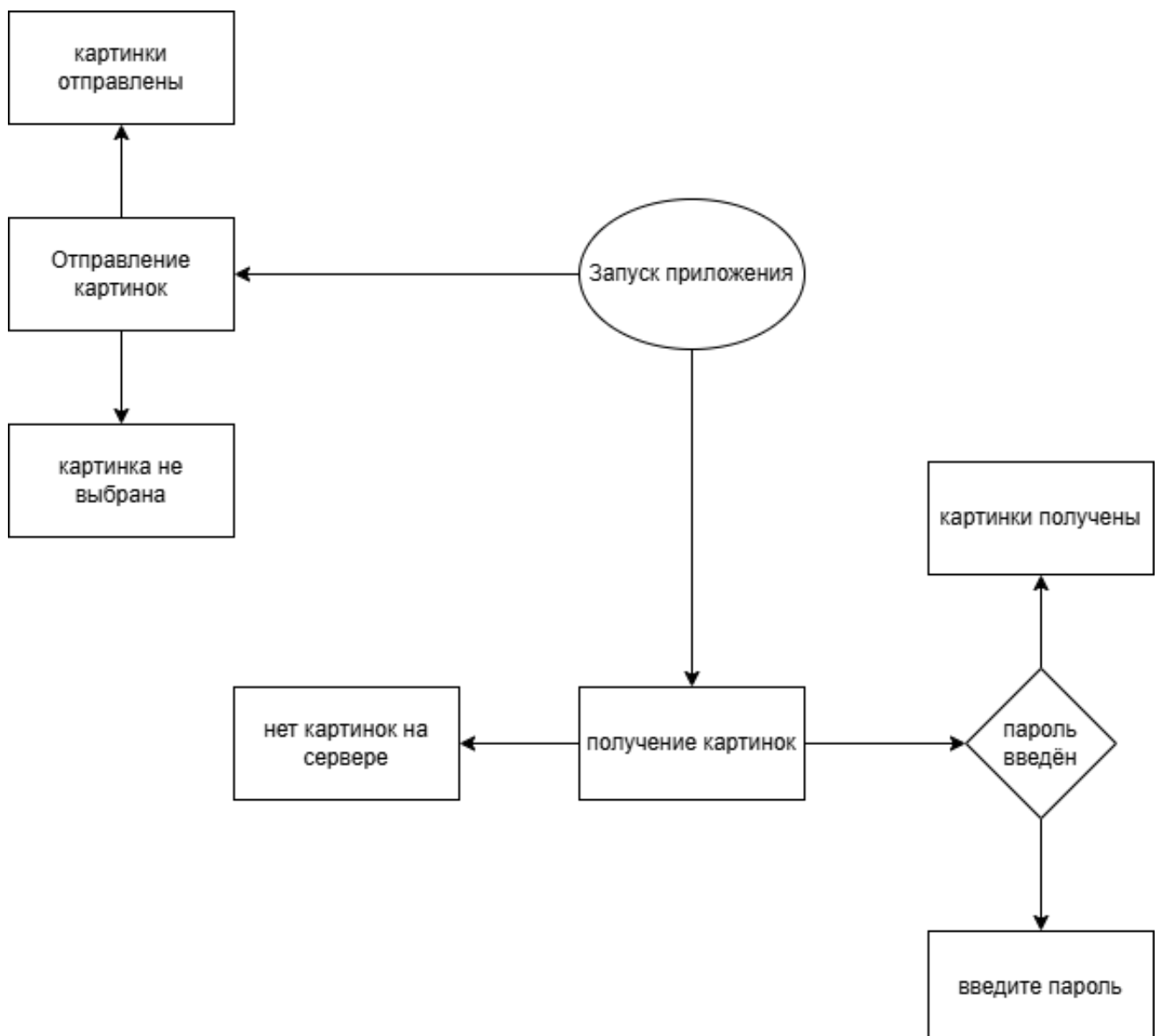
Серверное приложение использует Django допускает два типа запросов POST и GET
post запрос должен отправлять на сервер изображения для записи на хранилище сервера
get запрос позволяет пользователю получить изображения сохранённые на сервере

Клиентское приложение использует PyQt5 и имеет простой понятный интерфейс
позволяющий пользователю выбрать изображение для отправки и отправить его на сервер
post запросом также клиентское приложение может отправлять get запрос чтобы получить
все изображения с сервера и отобразить их

2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Отсутствует

3.СХЕМА РАЗРАБОТАННОЙ ПРОГРАММЫ



4.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

- 1.Запуск сервера и клиента
- 2.Выбор картинки для отправления
 - 2.1.Отправление выбранного изображения на сервер
- 3.Ввод пароля для получения изображений
 - 3.1.Получение изображений с сервера

5.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

- 1.Файлы изображения
- 2.Пароль для доступа к изображениям

6.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

- 1.Отображаемые в клиенте приложения

7.КАК ПОЛЬЗОВАТЬСЯ ВАШЕЙ ПРОГРАММОЙ

- 1.Запуск программы
- 2.Выбрать изображение для отправки
- 3.Отправить изображение на сервер
- 4.Ввести пароль в поле для пароля
- 5.Получить изображения с сервера

8.СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ

Программа может работать вне зависимости от системы

9.ЛИСТИНГ

1. Код клиента

```
import os
2. import sys
3. import requests
4. from PIL import Image
5. from PyQt5.QtWidgets import (
6.     QApplication, QWidget, QVBoxLayout,
7.     QHBoxLayout, QPushButton, QTextEdit,
8.     QLabel, QGroupBox, QLineEdit, QFileDialog,
9.     QListWidget, QListWidgetItem
10.
11.)
12. from PyQt5.QtGui import QImage, QPixmap
13. import json
14. from PIL import Image
15. from PyQt5 import QtCore
16.
17. class MyWindow(QWidget):
18.     def __init__(self):
```

```
19.     super().__init__()
20.     self.initUI()
21.
22.     def initUI(self):
23.         over_main_layout = QVBoxLayout()
24.         main_layout = QHBoxLayout()
25.         over_main_layout.addLayout(main_layout)
26.
27.         self.button1 = QPushButton(f"отправить")
28.         self.button2 = QPushButton(f"получить")
29.         self.button1.clicked.connect(self.change_text1)
30.         self.button2.clicked.connect(self.change_text2)
31.         self.images_group = QListWidget()
32.         self.images_group.itemClicked.connect(self.item_clicked)
33.         self.image = QLabel()
34.         self.image_button = QPushButton()
35.         self.image_button.setText('выбрать изображение для отправки')
36.         self.image_button.clicked.connect(self.pick_file)
37.         self.widget3 = QLabel()
38.         self.security = QGroupBox("введите пароль - (1)")
39.         self.security_layout = QVBoxLayout()
40.         self.security.setLayout(self.security_layout)
41.         self.password_field = QLineEdit()
42.         self.security_layout.addWidget(self.password_field)
43.         self.security_layout.addWidget(self.widget3)
44.         self.security_layout.addWidget(self.image_button)
45.         self.security_layout.addWidget(self.images_group)
46.         self.security_layout.addWidget(self.image)
47.         self.password = '1'
48.         # self.widget1 = self.create_widget("отправить на сервер", 1)
49.         # self.widget2 = self.create_widget("получить с сервера", 2)
50.
51.         self.pixmap = QPixmap('')
52.
53.         main_layout.addWidget(self.button1)
54.         main_layout.addWidget(self.button2)
55.
56.         over_main_layout.addWidget(self.security)
57.         self.setLayout(over_main_layout)
58.         self.setWindowTitle('Два виджета с кнопками и текстом')
59.         self.setGeometry(300, 300, 600, 600)
60.
61.     def create_widget(self, title, widget_num):
62.         group_box = QGroupBox(title)
63.         layout = QVBoxLayout()
64.         if widget_num == 1:
65.             self.text_edit1 = QTextEdit()
66.             self.text_edit1.setPlaceholderText(f"Текст для {title}")
67.             layout.addWidget(self.text_edit1)
68.         else:
69.             self.text_edit2 = QTextEdit()
70.             self.text_edit2.setPlaceholderText(f"Текст для {title}")
```

```

71.         layout.addWidget(self.text_edit2)
72.
73.         button = QPushButton(f"Изменить текст в {title}")
74.
75.         if widget_num == 1:
76.             button.clicked.connect(self.change_text1)
77.         else:
78.             button.clicked.connect(self.change_text2)
79.
80.         layout.addWidget(button)
81.         group_box.setLayout(layout)
82.
83.         return group_box
84. # , 'password' : self.password_field.text()}
85. def change_text1(self):
86.     try:
87.         if self.password_field.text() != self.password:
88.             self.widget3.setText('введите пароль')
89.             return None
90.         with open (self.image_path[0], 'rb') as f:
91.             print(f)
92.             print('-----')
93.             r = requests.post("http://127.0.0.1:8000/main/images/", files =
{"image": f})
94.             self.widget3.setText('изображение отправлено')
95.             print(r.text)
96.         except:
97.             self.widget3.setText('выберите изображение для отправки')
98.             # self.widget3.setText(r.text)
99.
100.    def change_text2(self):
101.        r = requests.get("http://127.0.0.1:8000/main/images/")
102.        try:
103.            if not json.loads(r.content):
104.                self.widget3.setText('на сервере пусто')
105.                r1 = requests.get(json.loads(r.content)[0]['image'])
106.            except:
107.                self.widget3.setText('ошибка получения(вероятно на сервере пусто)')
108.            for i in json.loads(r.content):
109.                x = requests.get(i['image'])
110.                self.pixmap.loadFromData(x.content)
111.                scaled_pixmap = self.pixmap.scaled(200, 200,
QtCore.Qt.KeepAspectRatio)
112.                self.image.setPixmap(scaled_pixmap)
113.                QListWidgetItem(i['image'], self.images_group)
114.                # self.text_edit2.setText(r.text)
115.        def item_clicked(self, item):
116.            print(item.text())
117.            x = requests.get(item.text())
118.            self.pixmap.loadFromData(x.content)
119.            scaled_pixmap = self.pixmap.scaled(200, 200,
QtCore.Qt.KeepAspectRatio)

```

```

120.         self.image.setPixmap(scaled_pixmap)
121.     def pick_file(self):
122.         self.image_path = QFileDialog().getOpenFileName(
123.             self,
124.             'выберите изображение',
125.             '',
126.             'image (*.png *.jpg)'
127.         )
128.         self.widget3.setText('изображение выбрано')
129.         # self.image = Image.open(fp=self.image_path[0])
130.         # print(self.image)
131.         # self.image.setNameFilter("image (*.png *.jpg)")
132.
133. if __name__ == '__main__':
134.     os.system('python project/run.py')
135.     app = QApplication(sys.argv)
136.     window = MyWindow()
137.     window.show()
138.     sys.exit(app.exec_())

```

2.Код Сервера

"""

Django settings for project project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see

<https://docs.djangoproject.com/en/5.2/topics/settings/>

For the full list of settings and their values, see

<https://docs.djangoproject.com/en/5.2/ref/settings/>

"""

```
from pathlib import Path
```

```
# Build paths inside the project like this: BASE_DIR / 'subdir'.
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
# Quick-start development settings - unsuitable for production
```

```
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/
```

```
# SECURITY WARNING: keep the secret key used in production secret!
```

```
SECRET_KEY = 'django-insecure-!b%tpw-*)#8gsb0q+c_dok9m-!cymkto-45uvp5+x+)ia9yvuk'
```

```
# SECURITY WARNING: don't run with debug turned on in production!
```

```
DEBUG = True
```

```
ALLOWED_HOSTS = []
```

```

# Application definition

APPEND_SLASH = False
CSRF_COOKIE_SECURE = False
CSRF_COOKIE_HTTPONLY = False
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'image_sender',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    # 'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'project.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {

```



```
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```